Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Summer, Year:2026), B.Sc. in CSE (Day)

Lab Report NO : 4
Course Title: Artificial Intelligence Lab
Course Code: CSE 316 Section: 241-D2

Lab Experiment Name: Implementation of N-Superqueen Problem.

Student Details

	Name	ID
1.	Md. REAHOON ZANNAH	223002038

Lab Date : 03-08-2026
Submission Date : 10-08-2026
Course Teacher's Name : Sian Ashsad

Lab Report Status

Marks: ……. **Signature:.....**
Comments:..... **Date:.....**

1. TITLE OF THE LAB REPORT EXPERIMENT

Implementation of N-Superqueen Problem.

2. OBJECTIVES

- To understand the basic concept of Constraint Satisfaction Problems (CSP).
- To understand the N-Superqueen (Amazon) Problem and its constraints.
- To implement the N-Superqueen problem using the Backtracking Algorithm in Python.
- To ensure that no two Superqueens attack each other.
- To find and display all possible solutions for a given value of N.
- To understand how backtracking explores possible positions and removes invalid assignments.
- To analyze the performance and limitations of the backtracking approach.

3. PROCEDURE

❖ Step of The Problem:

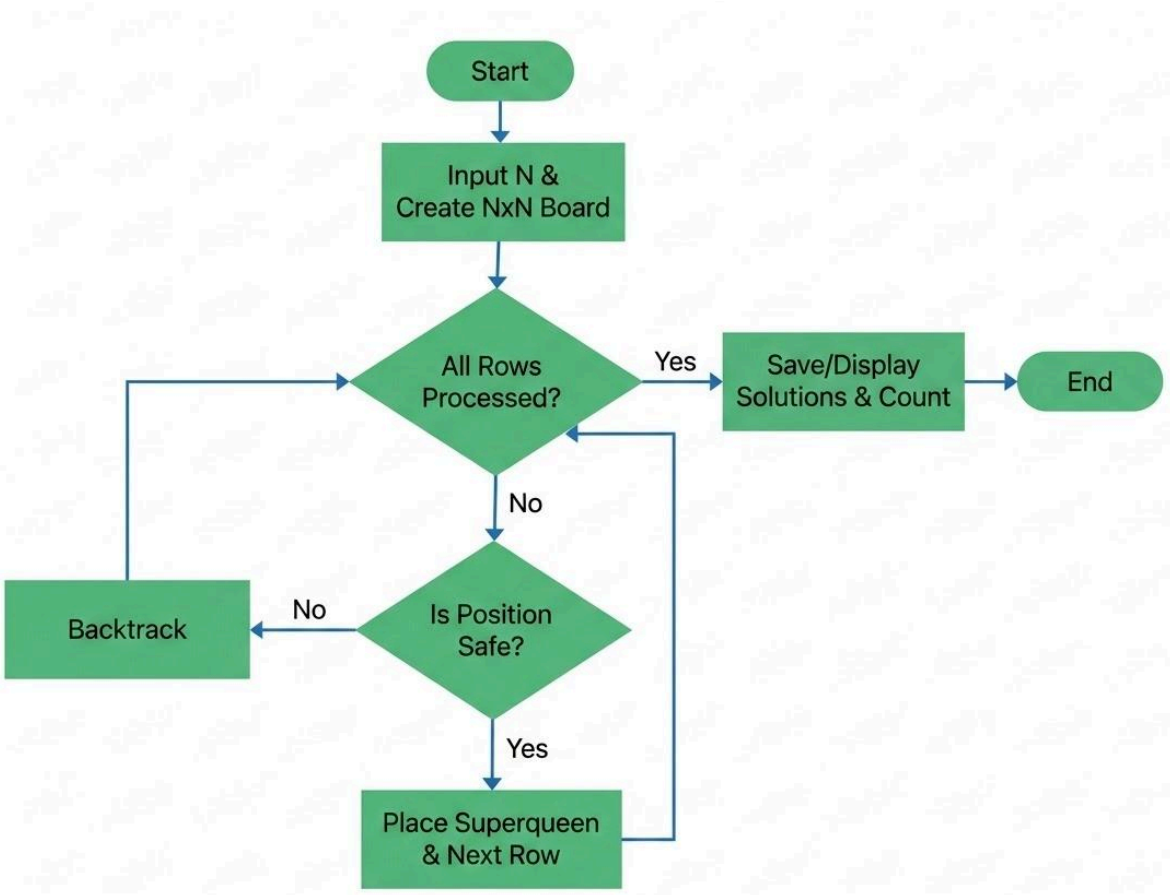

Fig: 4.1: Procedure flow chart.

4. IMPLEMENTATION:

a) N-super queen Problem:

```
def solve_superqueens(n):
    solutions = []

    # queen positions
    queens = [-1] * n

    # Used columns and diagonals
    columns = set()
    diagonal1 = set()   # row - column
    diagonal2 = set()   # row + column

    def is_safe(row, col):
        # Queen attacks
        if col in columns:
            return False

        if row - col in diagonal1:
            return False

        if row + col in diagonal2:
            return False

        # Knight attacks
        for previous_row in range(row):
            previous_col = queens[previous_row]

            row_difference = abs(row - previous_row)
            col_difference = abs(col - previous_col)

            if ((row_difference == 1 and col_difference == 2) or
                (row_difference == 2 and col_difference == 1)):
                return False

        return True

    def backtrack(row):
        # All N Superqueens have been placed
        if row == n:
            solutions.append(queens.copy())
            return

        # Try every column in this row
        for col in range(n):
```

```
            if is_safe(row, col):

                queens[row] = col

                columns.add(col)
                diagonal1.add(row - col)
                diagonal2.add(row + col)

                backtrack(row + 1)

                # Backtrack
                columns.remove(col)
                diagonal1.remove(row - col)
                diagonal2.remove(row + col)

                queens[row] = -1

    backtrack(0)

    return solutions



N = int(input("Enter N: "))

solutions = solve_superqueens(N)

print("\nTotal solutions:", len(solutions))

for number, solution in enumerate(solutions, 1):

    print(f"\nSolution {number}:")

    for row in range(N):
        for col in range(N):

            if solution[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")

        print()
```

5. OUTPUT

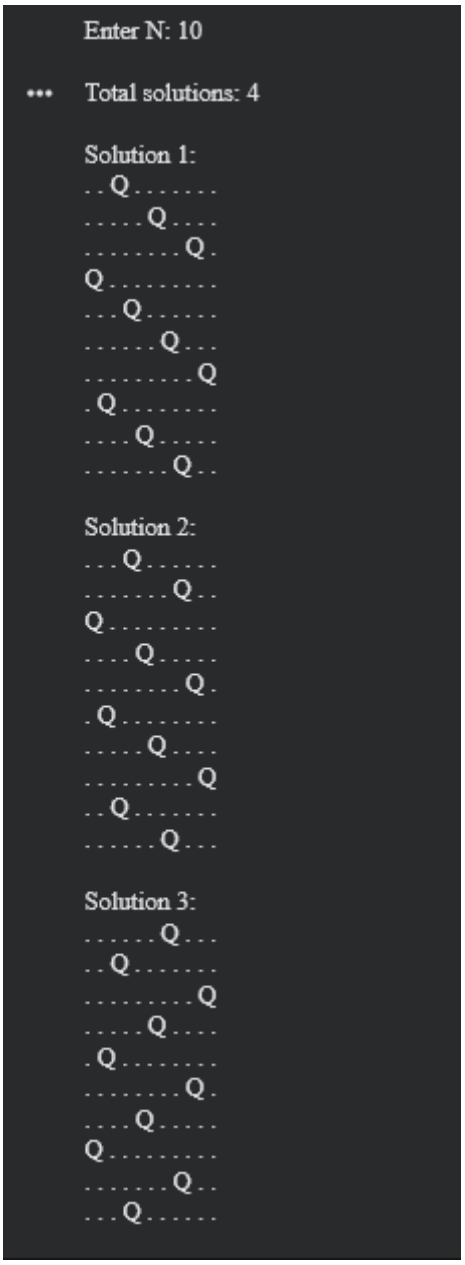

Fig-4.2: Result of (a).

6. ANALYSIS AND DISCUSSION

The N-Superqueen problem is a Constraint Satisfaction Problem in which N Superqueens must be assigned to an N × N chessboard while satisfying several constraints.

The implemented backtracking algorithm successfully explores the possible positions of Superqueens and rejects a position whenever it violates any Queen or Knight movement constraint.

Unlike the traditional N-Queen problem, the N-Superqueen problem is more restrictive because each piece has additional Knight-type attacks. Therefore, a position that would be valid for the traditional N-Queen problem may become invalid for the Superqueen problem.

7. SUMMARY

The N-Superqueen problem was successfully implemented using CSP and the Backtracking Algorithm. The Superqueen combines the movement of a Queen and a Knight, so all placements must satisfy column, diagonal, and Knight-movement constraints. The algorithm checks each position and backtracks whenever a conflict occurs. For N = 8, the program found 0 valid solutions. Overall, the experiment demonstrates how CSP and Backtracking can solve complex constraint-based problems.

8. GITHUB REPOSITORY:

Link: https://github.com/pro382r/GUB-Academic/blob/main/AI-lab/lr4/n-super-queen.ipynb

9. COLAB FILE:

Link: 223002038-AIL-lr4.ipynb